Human Verification: `Composition`

Container Composition $\triangleleft$ and its Monoidal Structure

Machine-generated from `lean/Composition.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Composition` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean.

The module equips the category **Cont** of containers with a **composition product** $\triangleleft$ whose extension satisfies $[\![G \triangleleft F]\!] = [\![G]\!] \circ [\![F]\!]$. The unit is the identity container I. The main results are the left and right unitors and the associator, each proved as an explicit isomorphism of containers. A $\triangleleft$-comonoid is exactly a directed container.

Contents

1 Overview **1**

2 The composition product **2**
2.1 Definition: `Container.comp` . . . 2
2.2 Definition: `Container.I` . . . 2

3 Extension isomorphism **2**
3.1 Definition: `Container.compExt` and `Container.compExtInv` . . . 2
3.2 Theorems: `compExtInv_compExt` and `compExt_compExtInv` . . . 3

4 Container isomorphisms **3**
4.1 Structure: `ContainerIso` . . . 3

5 Monoidal coherences **4**
5.1 Definition: `Container.leftUnitor` . . . 4
5.2 Definition: `Container.rightUnitor` . . . 4
5.3 Definition: `Container.associator` . . . 5

6 The compositional payoff **5**

7 Translation conventions **6**

1 Overview

For containers $F = S \triangleleft P$ and $G = T \triangleleft Q$, their **composition** $G \triangleleft F$ ("G outside, F inside") has

- shapes $\Sigma_{t:T}(Q(t) \to S)$ — an outer G-shape t together with an assignment of an inner F-shape to each G-position; and

- positions at (t, f) given by $\Sigma_{q:Q(t)} P(f(q))$ — pick an outer position q, then a position in the inner shape $f(q)$.

The key fact is that $[\![G \triangleleft F]\!]X \cong [\![G]\!]([\![F]\!]X)$, made definitional by the `compExt`/`compExtInv` pair.

2 The composition product

2.1 Definition: `Container.comp`

Definition 2.1 (Container composition)**.** For containers G and F, their composition $G \triangleleft F$ is the container with

$$\mathrm{Shape}(G \triangleleft F) = \Sigma_{t:G.\mathrm{Shape}} (G.\mathrm{Pos}(t) \to F.\mathrm{Shape}),$$
$$\mathrm{Pos}(G \triangleleft F)(t, f) = \Sigma_{q:G.\mathrm{Pos}(t)} F.\mathrm{Pos}(f(q)).$$

Lean.

```
def Container.comp (G F : Container) : Container where
  Shape := (t : G.Shape) * (G.Pos t → F.Shape)
  Pos := fun tf ⇒ (q : G.Pos tf.1) * F.Pos (tf.2 q)

scoped infixr:70 "␣comp␣" ⇒ Container.comp
-- (written with the unicode diamond operator in Lean source)
```

Haskell.

```
-- G `outside` F:
data CompShape g p_g f = CompShape g (p_g → f)
-- Positions at (t, assign):
data CompPos p_g p_f t assign = CompPos (p_g t) (p_f (assign (p_g t)))
-- Approximate (non-dependent):
data Comp g p_g f p_f x = Comp g (p_g → f) (p_g → p_f → x)
```

Fidelity note. The shape and position types depend on one another in a genuinely dependent way (P_F is indexed by the value of the inner shape function $f(q)$). Haskell cannot express this; the rendering above is an approximation that loses the dependency.

2.2 Definition: `Container.I`

Definition 2.2 (Identity container)**.** The **identity container** I has one shape (the unit type) and one position. Its extension is the identity functor: $[\![I]\!]X = X$.

Lean.

```
def Container.I : Container where
  Shape := Unit
  Pos := fun _ ⇒ Unit
```

Haskell.

```
-- Identity functor: Id x = x.
-- As a container: one shape (Unit), one position (Unit).
newtype Id x = Id x
```

3 Extension isomorphism

3.1 Definition: `Container.compExt` and `Container.compExtInv`

Definition 3.1 (Extension isomorphism)**.** There is a natural bijection

$$[\![G \triangleleft F]\!]X \;\cong\; [\![G]\!]([\![F]\!]X),$$

given by:

$$\mathrm{compExt}\,\langle (t, f), \ell\rangle = \langle t,\ q \mapsto \langle f(q),\ r \mapsto \ell(q, r)\rangle\rangle,$$
$$\mathrm{compExtInv}\,\langle t, m\rangle = \langle (t,\ q \mapsto m(q).1),\ (q, r) \mapsto m(q).2(r)\rangle.$$

Lean.

```
def Container.compExt {G F : Container} {X : Type} :
    Ext (G comp F) X → Ext G (Ext F X) :=
  fun p ⇒ ⟨p.1.1, fun q ⇒ ⟨p.1.2 q, fun r ⇒ p.2 ⟨q, r⟩⟩⟩

def Container.compExtInv {G F : Container} {X : Type} :
    Ext G (Ext F X) → Ext (G comp F) X :=
  fun p ⇒ ⟨⟨p.1, fun q ⇒ (p.2 q).1⟩, fun qr ⇒ (p.2 qr.1).2 qr.2⟩
```

Haskell.

```
compExt :: Comp g p_g f p_f x → g → (p_g → (f, p_f → x))
compExt (Comp t assign lab) = (t, \q → (assign q, \r → lab q r))

compExtInv :: (g, p_g → (f, p_f → x)) → Comp g p_g f p_f x
compExtInv (t, m) = Comp t (fst . m) (\q r → snd (m q) r)
```

3.2 Theorems: `compExtInv_compExt` and `compExt_compExtInv`

Theorem 3.2 (Extension isomorphism is an isomorphism)**.** The maps `compExt` and `compExtInv` are mutually inverse:

$$\mathrm{compExtInv} \circ \mathrm{compExt} = \mathrm{id},$$
$$\mathrm{compExt} \circ \mathrm{compExtInv} = \mathrm{id}.$$

Both round trips hold definitionally (proved by `rfl` after `cases`), because the position fibres agree definitionally.

Lean.

```
theorem Container.compExtInv_compExt {G F : Container} {X : Type}
    (p : Ext (G comp F) X) :
    Container.compExtInv (Container.compExt p) = p

theorem Container.compExt_compExtInv {G F : Container} {X : Type}
    (p : Ext G (Ext F X)) :
    Container.compExt (Container.compExtInv p) = p
```

Haskell.

```
-- compExtInv . compExt = id   (and vice versa)
-- Both hold by the eta-laws for pairs.
```

4 Container isomorphisms

4.1 Structure: `ContainerIso`

Definition 4.1 (Container isomorphism)**.** An **isomorphism of containers** $C \cong D$ is a pair of container morphisms $\varphi : C \to D$ and $\psi : D \to C$ satisfying:

$$\varphi \circ \psi = \mathrm{id}_C \quad \text{and} \quad \psi \circ \varphi = \mathrm{id}_D.$$

(Composition here is morphism composition in **Cont**.)

Lean.
```
structure ContainerIso (C D : Container) where
  hom : ContainerMorphism C D
  inv : ContainerMorphism D C
  hom_inv : hom.comp inv = ContainerMorphism.id C
  inv_hom : inv.comp hom = ContainerMorphism.id D
```

Haskell.
```
data ContainerIso c d = ContainerIso
  { hom     :: Morph c d
  , inv     :: Morph d c
  , hom_inv :: hom `compMorph` inv == idMorph c    -- laws
  , inv_hom :: inv `compMorph` hom == idMorph d
  }
```

5 Monoidal coherences

5.1 Definition: `Container.leftUnitor`

Definition 5.1 (Left unitor)**.** There is a canonical isomorphism of containers

$$\lambda_F : I \triangleleft F \;\cong\; F.$$

A shape of $I \triangleleft F$ is $(\langle\rangle, f)$ with $f : \mathrm{Unit} \to F.\mathrm{Shape}$; the bijection with $F.\mathrm{Shape}$ sends $f \mapsto f(\langle\rangle)$ and $s \mapsto (\lambda _ \to s)$. Positions collapse by the same Unit-projection.

Lean.
```
def Container.leftUnitor (F : Container) :
    ContainerIso (Container.I comp F) F where
  hom :=
    { onShapes := fun tf ⇒ tf.2 ()
      onPos := fun _ p ⇒ ⟨(), p⟩ }
  inv :=
    { onShapes := fun s ⇒ ⟨(), fun _ ⇒ s⟩
      onPos := fun _ qp ⇒ qp.2 }
  hom_inv := rfl
  inv_hom := rfl
```

Haskell.
```
leftUnitor :: ContainerIso (I `Comp` F) F
-- hom: ((), f) |→ f (); pos: p |→ ((), p)
-- inv: s     |→ ((), const s); pos: (q,p) |→ p
```

5.2 Definition: `Container.rightUnitor`

Definition 5.2 (Right unitor)**.** There is a canonical isomorphism of containers

$$\rho_F : F \triangleleft I \;\cong\; F.$$

A shape of $F \triangleleft I$ is (s, g) with $g : F.\mathrm{Pos}(s) \to \mathrm{Unit}$; the bijection projects onto s. Positions collapse $\Sigma_{p:F.\mathrm{Pos}(s)}\mathrm{Unit} \cong F.\mathrm{Pos}(s)$.

Lean.
```
def Container.rightUnitor (F : Container) :
    ContainerIso (F comp Container.I) F where
  hom :=
    { onShapes := fun sg ⇒ sg.1
      onPos := fun _ p ⇒ ⟨p, ()⟩ }
```

```
  inv :=
    { onShapes := fun s ⇒ ⟨s, fun _ ⇒ ()⟩
      onPos := fun _ pu ⇒ pu.1 }
  hom_inv := rfl
  inv_hom := rfl
```

Haskell.

```
rightUnitor :: ContainerIso (F `Comp` I) F
-- hom: (s, g) |→ s;    pos: p |→ (p, ())
-- inv: s      |→ (s, const ()); pos: (p,()) |→ p
```

5.3 Definition: `Container.associator`

Definition 5.3 (Associator)**.** There is a canonical isomorphism of containers

$$\alpha_{H,G,F} : (H \triangleleft G) \triangleleft F \;\cong\; H \triangleleft (G \triangleleft F).$$

On shapes it curries/uncurries the inner shape assignment: a shape $((a, g), k)$ on the left (where $k : \Sigma_{u:H.\mathrm{Pos}(a)} G.\mathrm{Pos}(g(u)) \to F.\mathrm{Shape}$) corresponds to (a, m) on the right (where $m(u) = \langle g(u),\, v \mapsto k(u, v)\rangle$). On positions it re-associates the nested dependent sum: $((u, v), p) \leftrightarrow (u, (v, p))$.

Crucially, the position fibre $F.\mathrm{Pos}((m\,u).2\,v)$ equals $F.\mathrm{Pos}(k(u, v))$ *definitionally*, so no transport is needed.

Lean.

```
def Container.associator (H G F : Container) :
    ContainerIso ((H comp G) comp F) (H comp (G comp F)) where
  hom :=
    { onShapes := fun xk ⇒
        ⟨xk.1.1, fun u ⇒ ⟨xk.1.2 u, fun v ⇒ xk.2 ⟨u, v⟩⟩⟩
      onPos := fun _ uvp ⇒ ⟨⟨uvp.1, uvp.2.1⟩, uvp.2.2⟩ }
  inv :=
    { onShapes := fun am ⇒
        ⟨⟨am.1, fun u ⇒ (am.2 u).1⟩, fun y ⇒ (am.2 y.1).2 y.2⟩
      onPos := fun _ yp ⇒ ⟨yp.1.1, yp.1.2, yp.2⟩ }
  hom_inv := ...  -- ContainerMorphism.ext' rfl (cases + rfl)
  inv_hom := ...  -- ContainerMorphism.ext' rfl (cases + rfl)
```

Haskell.

```
associator :: ContainerIso ((H `Comp` G) `Comp` F) (H `Comp` (G `Comp`
    F))
-- hom shapes: ((a,g),k) |→ (a, \u → (g u, \v → k (u,v)))
-- hom pos:    ((u,v),p) |→ (u,(v,p))
-- inv shapes: (a,m)     |→ ((a, fst . m), \(u,v) → snd (m u) v)
-- inv pos:    (u,(v,p)) |→ ((u,v),p)
```

6 The compositional payoff

The fact that $(\mathbf{Cont}, \triangleleft, I)$ is a monoidal category (with the above coherences) means that a **$\triangleleft$-comonoid** in **Cont** — a container C equipped with a counit $C \to I$ and a comultiplication $C \to C \triangleleft C$ satisfying the comonoid laws — is exactly a **directed container** (equivalently, a comonad on **Set** whose underlying functor is a container). The comonoid laws unfold to the five directed-container axioms (D1–D5) formalised in `Containers.Directed`.

7 Translation conventions

1. **Dependent types.** The position type in $G \triangleleft F$ depends on the shape value. Haskell approximations drop this dependency.
2. **Definitional equalities.** The round trips for `compExt` / `compExtInv` and for the associator both hold *definitionally* in Lean (proved by `rfl` after `cases`). This is a key feature of the Lean 4 type theory, not available in most proof assistants.
3. **Unicode notation.** Lean uses the diamond operator $\triangleleft$ for container composition. The listings use `comp` as an ASCII placeholder.
4. **Proof bodies omitted.** Only statements are presented here.